Cryptography and Network Security Integrated Situation Report

Abstract

This report documents a risk assessment, an AES-256-GCM file encryption and decryption utility, SHA-256 integrity checking, and a routed nftables policy template for an authorized laboratory. Network identifiers and service details were not supplied in the assignment, so they remain marked placeholders. Laboratory firewall tests have not been executed; expected outcomes are distinguished from actual evidence.

1. Scope and assets

The scenario concerns student records on a central server and file transfers between two campuses. It identifies weak staff passwords, outdated software, unencrypted transfers, guest reachability to the records server, and repeated attempts from an unfamiliar external address. No real student data or production network was used in preparing this project.

Asset	Vulnerability or threat	Possible consequence
Student records and server	Guest access and unfamiliar external attempts	Unauthorized disclosure, alteration, loss, or service disruption
Staff accounts and systems	Weak passwords and outdated software	Account compromise, exploitation, malware, unauthorized access
Inter-campus files and transfer service	Unencrypted transfers	Interception or modification in transit

1. Risk ranking and controls

Ratings are qualitative estimates based only on the scenario, not measured incident data.

Rank / risk	Likelihood	Impact	Reason and recommended control
1. Guest or external access to records server	High	High	Guest reachability and repeated external attempts are stated. Apply default-deny filtering, block guest and other inbound sources, allow only authorized staff to the assessor-selected service, and review logs.
2. Staff account compromise or software exploitation	Medium	High	Weak passwords and outdated software are confirmed, but no compromise is reported. Enforce unique strong passwords and MFA where available, patch supported systems, minimise privileges, and review access.

3. Interception or tampering in transit	Medium	High	Transfers are unencrypted, though no interception is reported. Use institution-approved authenticated encryption for transfer and verify integrity over a trusted channel.

2. Encryption and integrity toolkit

The Python CLI uses AES-256-GCM from the cryptography package. A fresh 96-bit nonce is generated for each encryption. The output has a format header followed by the nonce and authenticated ciphertext; decryption rejects malformed data, wrong keys, and altered ciphertext. Inputs and outputs must be different files, and the tool refuses to overwrite outputs. A random 256-bit key is generated separately and saved outside the repository with restricted permissions where supported.

The hash command calculates SHA-256 and can save a JSON baseline. The verify command compares a later file digest with that baseline and reports match or mismatch. The baseline must be protected independently: an unkeyed hash alone does not establish who created it.

Reproducible evidence

The synthetic checks run for this project are recorded in crypto_tests.md: key generation, encryption, decryption with byte-for-byte round trip, matching and changed-file SHA-256 verification, missing input, and malformed ciphertext. The changed-file check returned exit status 1 as expected; malformed inputs returned concise errors with exit status 2. The CLI also handles malformed key files, invalid hash records, and overwrite attempts with a concise error rather than an unhandled exception.

3. Firewall policy and connection testing

The included firewall.nft.template is a routed Linux nftables example with a default-drop forwarding policy. It blocks guest traffic to the records server before accepting established traffic, allows the staff CIDR to the server only on the assessor-approved service port, denies other sources to that service, and drops remaining forwarded packets.

The assignment does not provide the server address, staff/guest CIDRs, gateway interface, or approved service, transport protocol, and port. The template cannot safely be applied until the authorized administrator supplies and reviews those values. Confirm that the approved protocol is supported by the nftables transport-header expression in the lab version; adapt the rules if it is not, if the server is not routed through this gateway, or if the lab uses another firewall platform.

Test	Expected outcome	Actual outcome
Staff client to approved service	Permitted while service is listening	Not executed; requires authorized lab and supplied values
Guest client to approved service	Blocked	Not executed; requires authorized lab and supplied values
Other source to approved service	Blocked	Not executed; requires authorized lab and supplied values

Detailed command templates and the safe execution procedure are in filter_tests.md. No firewall rules were applied and no network tests were run for this project build. Expected results are policy expectations, not observed results.

4. Conclusion

The project provides a structured assessment of the three scenario risks, an authenticated file encryption tool with SHA-256 baseline checks, and a default-deny firewall template with a reproducible three-case test plan. Completion of the live firewall portion requires the assessor's exact network and service values plus authorized lab access. Add actual test outcomes and student-identifying information before final submission.

GitHub Repository URL: https://github.com/olivieryeada8-del/cryptography-network-security-exam.git